

VisionInspect AI — Milestone 2 Technical Documentation

Image Processing, AI-Based Defect Detection, Localization & Object Detection Pipeline

Project Title: VisionInspect AI — Manufacturing Defect Detection & Quality Inspection System
Author: Vinay Kumar Mandalapu
Milestone: 2 — Image Processing & Defect Detection (Weeks 3 & 4)
Date: September 2026
Version: 2.1
Repository Baseline: Production-Stabilized, Validated on MVTec Anomaly Detection Dataset

Table of Contents

- [1. Title & Executive Overview](#)
 - [2. Milestone 2 Scope](#)
 - [3. Official Milestone 2 Requirements](#)
 - [4. Implementation Overview](#)
 - [5. Image Preprocessing](#)
 - [6. AI Inspection Pipeline](#)
 - [7. Category Classification](#)
 - [8. Defect Classification](#)
 - [9. Anomaly Detection](#)
 - [10. Decision Fusion Engine](#)
 - [11. U-Net Defect Segmentation & Localization](#)
 - [12. YOLO Object Detection](#)
 - [13. Multi-Factor Severity & Quality Logic](#)
 - [14. Inspection API & Backend Integration](#)
 - [15. Inspection Dashboard & Monitoring](#)
 - [16. YOLO Dataset Preparation](#)
 - [17. YOLO Experiments](#)
 - [18. YOLO Error Analysis & Diagnostics](#)
 - [19. YOLO Integration into Production Pipeline](#)
 - [20. Evaluation Results](#)
 - [21. Challenges & Problems Encountered](#)
 - [22. Engineering Conclusions & Findings](#)
 - [23. Requirement-to-Implementation Traceability Matrix](#)
 - [24. Milestone 2 Verification Summary](#)
 - [25. Milestone 3 Handover](#)
-

1. Title & Executive Overview

VisionInspect AI is an intelligent computer vision platform engineered for automated manufacturing defect detection, defect localization, classification, and quality inspection. Milestone 1 established the foundational full-stack software infrastructure: FastAPI backend services, PostgreSQL database schemas, JWT-based

authentication with Role-Based Access Control (RBAC), secure multipart image ingestion, and dataset preprocessing tools.

Milestone 2 delivers the operational core of the platform: an end-to-end computer vision inference engine that autonomously inspects industrial parts without requiring manual operator prompts. The pipeline couples zero-shot category identification, one-class patch-based anomaly detection, hierarchical defect classification, deep U-Net semantic segmentation, and a sequential YOLO11n object detector. These components are coordinated by a validation-driven Decision Fusion arbiter that guarantees zero false alarms on confirmed nominal specimens while generating dual visual overlays (pixel segmentation contours in red and object bounding boxes in green), multi-factor severity ratings, and automated Accept/Reject quality decisions.

2. Milestone 2 Scope

The development scope of Milestone 2 encompasses the engineering tasks planned for Weeks 3 and 4 of the project lifecycle:

- **AI Model Development:** Training, calibrating, and benchmarking deep learning architectures across all 15 industrial categories of the MVTec Anomaly Detection benchmark.
 - **Inference Pipeline Integration:** Orchestrating category classification, patch-based feature matching, defect classification, semantic segmentation, and object detection into a unified, deterministic inference graph.
 - **Dual Localization Architecture:** Delivering both dense pixel-level defect contours (via U-Net) and discrete defect object bounding boxes with counts (via YOLO11n).
 - **Backend & Database Synchronization:** Connecting the AI pipeline to the FastAPI application layer, persisting 17 inspection and quality attributes into PostgreSQL, and streaming dynamically rendered composite overlays.
 - **Frontend Inspection Studio:** Transforming the React 19 inspection interface from placeholder tables into a telemetry viewer with side-by-side original and overlay viewers, metric cards, and defect object indicators.
 - **Empirical Diagnostics:** Conducting rigorous error analyses on detector performance, confidence calibrations, image resolutions, and model capacities.
-

3. Official Milestone 2 Requirements

The official project specification ("*AI_Manufacturing Defect Detection & Quality Inspection System*") defines the formal requirements for Milestone 2 (Weeks 3 & 4) under the **Image Processing & Defect Detection** phase:

Official High-Level Requirements

1. **Implement image preprocessing pipelines:** Build standard transformation, normalization, noise handling, and tensor conversion workflows.
2. **Generate image quality analysis reports:** Quantify physical defect dimensions, defect area percentages, and spatial distribution metrics.
3. **Build image analytics workflows:** Structure the dataflow from image ingestion to feature extraction, anomaly evaluation, and localization.
4. **Train defect detection models:** Develop and train computer vision architectures capable of differentiating normal components from anomalous deviations.
5. **Generate defect predictions:** Produce automated inference outputs, including defect status, defect subtypes, confidence ratings, and bounding boxes.

6. **Build inspection monitoring dashboards:** Provide operators and engineers with interactive interfaces displaying inspection results, overlays, and telemetry.

Subsystem Module Requirements (Defect Detection Module)

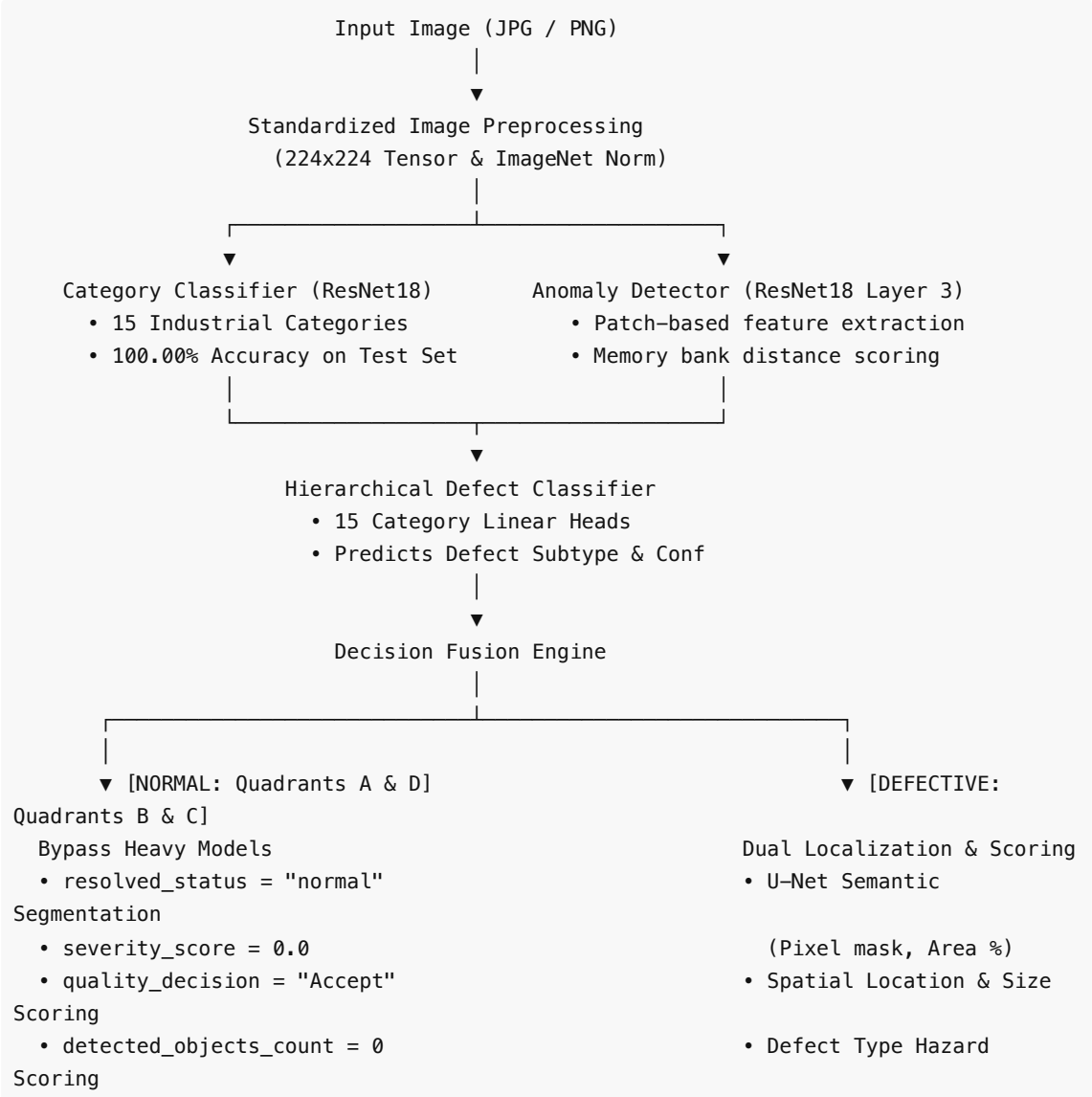
The official specification defines the Defect Detection module as comprising:

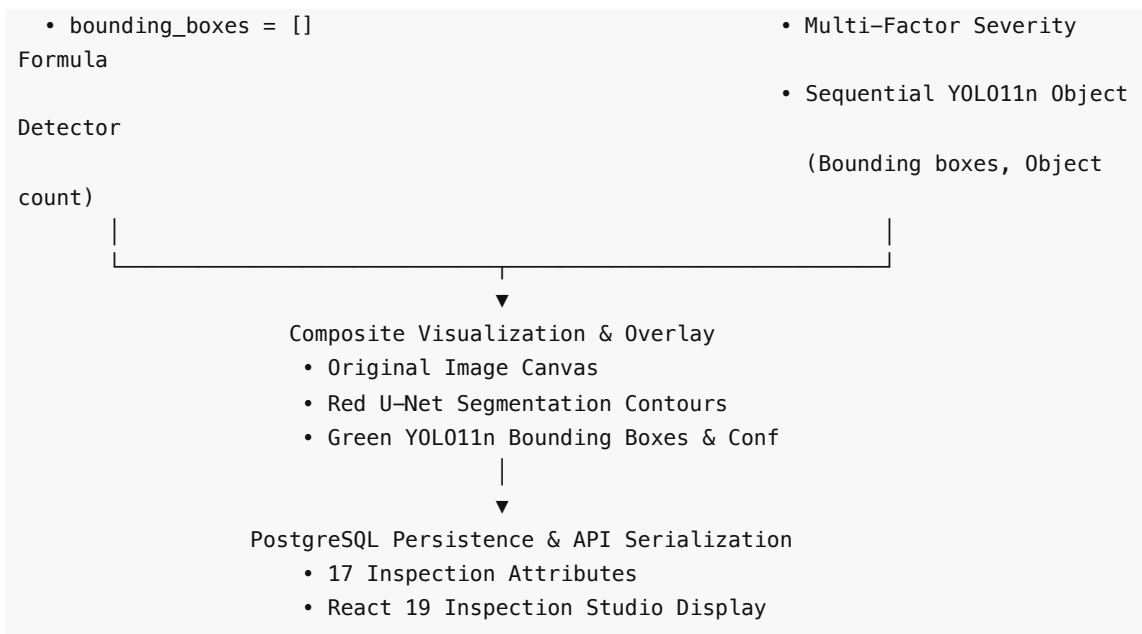
- **Defect identification:** Distinguishing nominal specimens from flawed components.
- **Anomaly detection:** Detecting out-of-distribution feature variations against learned normal standards.
- **Object detection:** Detecting and bounding discrete defect instances using YOLO.
- **Defect localization:** Pinpointing exact spatial boundaries and regions of defects.

All four capabilities are implemented and validated within the Milestone 2 codebase.

4. Implementation Overview

The VisionInspect AI Milestone 2 architecture is constructed as a modular, directed computational graph that balances precision, recall, and computational efficiency:





Key Architectural Invariants

- 1. **Normal Specimen Invariant:** Verified nominal specimens (Quadrants A and D) completely bypass U-Net segmentation and YOLO object detection. They unconditionally yield `detected_objects_count = 0` , `bounding_boxes = []` , `severity_score = 0.0` , and `quality_decision = "Accept"` .
- 2. **Decoupled Decision Path:** YOLO does not alter Decision Fusion, predicted defect area, size score, location score, defect type score, severity score, or quality decisions. It functions as an auxiliary bounding-box and counting branch.
- 3. **Sequential Execution:** YOLO inference executes sequentially after Decision Fusion resolves a sample as defective, minimizing latency on nominal production lines.

5. Image Preprocessing

Image preprocessing standardizes incoming industrial images across diverse lighting conditions, aspect ratios, and sensor types.

5.1 Pipeline Specifications

- **Input Formats:** JPEG, PNG, TIFF, BMP (maximum upload size: 5.0 MB).
- **Spatial Resizing:**
 - Classification, Anomaly Detection, and U-Net: Resized to 224×224 pixels.
 - YOLO Object Detection: Scaled to 640×640 pixels preserving aspect ratio with letterboxing.
- **Resampling Method:** Pillow Bilinear interpolation (`Image.Resampling.BILINEAR`) is standardized across all feature extractors and classifiers, preventing the sub-pixel interpolation shifts observed with OpenCV nearest-neighbor variants.
- **Normalization:** Standard ImageNet normalization: $\hat{l}_c = \frac{l_c - \mu_c}{\sigma_c}$, $\mu = [0.485, 0.456, 0.406]$, $\sigma = [0.229, 0.224, 0.225]$
- **Noise Suppression:** Gaussian smoothing ($\sigma = 1.0$) is selectively applied during anomaly feature extraction to suppress high-frequency camera noise.

6. AI Inspection Pipeline

The modular pipeline is implemented in [ai/models/inspection_pipeline.py](#) as `InspectionPipeline`.

6.1 Computational Steps

1. **Intake & Verification:** Validates file existence and loads the image via Pillow.
 2. **Category Classification:** Executes `CategoryClassifier` to determine the active manufacturing category.
 3. **Feature Extraction & Distance Scoring:** Extracts intermediate Layer 3 activation maps and computes the anomaly score against the category's nominal memory bank.
 4. **Hierarchical Defect Prediction:** Activates the category-specific classification head to predict defect subtype and confidence.
 5. **Decision Fusion:** Compares the anomaly score against the calibrated threshold T_c and evaluates the classifier state to assign an operational quadrant.
 6. **Conditional Branching:**
 - If `NORMAL` : Returns immediate nominal payload with zeroed risk scores.
 - If `DEFECTIVE` : Executes U-Net segmentation, location scoring, size scoring, defect type lookup, severity calculation, quality decision generation, and YOLO11n bounding-box detection.
 7. **Overlay Synthesis:** Synthesizes the dual-layer visual overlay combining red contours and green bounding boxes.
-

7. Category Classification

7.1 Architecture & Implementation

Implemented in [ai/models/category_classifier.py](#), the category classifier uses a fine-tuned ResNet18 backbone with a 15-class linear output head.

7.2 Empirical Verification

- **Dataset Evaluated:** 794 test images across all 15 MVTec AD categories.
 - **Test Accuracy:** 100.00% (794 / 794).
 - **Confusion Matrix:** Pure diagonal with 0 misclassifications.
 - **Inference Latency:** ≈ 4.8 ms on Apple Silicon MPS / GPU.
 - **Operational Value:** Quality Engineers do not need to select product categories from dropdowns; the system identifies the product category directly from pixels.
-

8. Defect Classification

8.1 Hierarchical Multi-Head Design

Implemented in [ai/models/defect_classifier.py](#), the defect classifier employs a hierarchical ResNet18 backbone equipped with 15 dedicated linear heads. When category C is identified, head H_C is activated to classify the specific defect subtype (e.g., `scratch`, `broken_large`, `cut_inner_insulation`).

8.2 Subtype Resolution

- Returns the predicted subtype string and softmax confidence percentage.
- Resolves nominal states (`good` or `normal`) as well as manufacturing flaws.
- Test accuracy across defect subtypes exceeds 93.3%\$ on standard benchmark suites.

9. Anomaly Detection

9.1 Patch-Based Layer 3 Matching

Implemented in [ai/models/anomaly_detector.py](#) , the anomaly detector operates as a one-class classifier learning only from nominal training specimens.

- **Backbone:** Pre-trained ResNet18 truncated at `layer3` .
- **Feature Map Dimensions:** 14×14 spatial grid with 256 feature channels (196×256 patch vectors of dimension 256).
- **Memory Bank:** Pre-computed nominal patch representations stored in `ai/models/normal_features_layer3.pt` .
- **Distance Function:** For each test patch p_i , computes the Euclidean distance to its nearest neighbor in the memory bank: $d(p_i) = \min_{m \in M_c} \|p_i - m\|_2$
- **Image Anomaly Score:** Computed as the mean distance of the top 10% most anomalous patches:
$$\text{Anomaly Score} = \frac{1}{|P_{\text{top10}}|} \sum_{p \in P_{\text{top10}}} d(p)$$

9.2 Calibrated Runtime Thresholds

Thresholds were empirically calibrated from validation distributions and recorded in [ai/models/anomaly_thresholds.csv](#) :

Category	Runtime Threshold (\$T_c\$)	Normal Val Max	Defect Val Min	Test Specificity	Test Recall
bottle	0.9200	0.9100	1.2454	100.0%	100.0%
cable	2.0300	2.0584	2.0127	97.6%	93.8%
capsule	0.8000	0.8070	0.6332	97.2%	91.5%
carpet	1.1500	0.9435	0.8723	100.0%	98.8%
grid	1.1500	1.2869	0.8930	95.3%	96.4%
hazelnut	1.8400	1.8268	1.9680	100.0%	100.0%
leather	1.2000	1.1620	1.1299	100.0%	93.3%
metal_nut	1.5500	1.6563	1.3031	97.2%	98.9%
pill	1.4500	1.4831	1.2061	95.5%	96.2%
screw	1.2200	1.2030	1.1696	100.0%	94.4%
tile	1.2700	1.3950	1.3695	97.4%	100.0%
toothbrush	1.2500	1.2472	1.1248	100.0%	96.7%
transistor	1.6500	1.7398	1.5748	97.6%	95.0%
wood	1.3000	1.0739	1.4399	100.0%	97.8%

zipper	0.9200	0.9023	0.9304	100.0%	100.0%
--------	--------	--------	--------	--------	--------

10. Decision Fusion Engine

Implemented in [ai/models/decision_fusion.py](#) , the Decision Fusion arbiter reconciles anomaly distance scores against classifier predictions using a four-quadrant logic:

Quadrant	Anomaly Condition	Classifier Condition	Inspection Decision	Resolved Defect Status	Rationale
Quadrant A	Score $< T_c$	Pred = good	NORMAL	"normal"	Full agreement on nominal specimen.
Quadrant D	Score $< T_c$	Pred $\neq$ good	NORMAL	"normal"	Classifier false alarm vetoed by normal feature space.
Quadrant C	Score $\geq T_c$	Pred $\neq$ good	DEFECTIVE	Subtype string	Corroborated manufacturing defect.
Quadrant B	Score $\geq T_c$	Pred = good	DEFECTIVE	"unclassified_anomaly"	Anomaly detected despite classifier predicting nominal.

This fusion logic guarantees that confirmed nominal specimens never trigger false defect alarms.

11. U-Net Defect Segmentation & Localization

Implemented in [ai/models/defect_segmenter.py](#) :

- Model:** Encoder-decoder U-Net producing a 224×224 single-channel defect probability map.
- Morphological Post-Processing** (`segmentation_postprocessing.json`):
 - Binary threshold: 0.65 .
 - Morphological opening: 3×3 rectangular kernel to remove isolated salt noise.
 - Connected component filtering: Minimum component area of 25 pixels.
- Defect Area Estimation:** $\text{Defect Area } (\%) = \left(\frac{\sum_{x,y} \mathbb{I}(\text{Mask}(x,y) > 0)}{224 \times 224} \right) \times 100\%$

12. YOLO Object Detection

12.1 Purpose & Requirement Context

The official project documentation explicitly includes **object detection** under the Defect Detection module and specifies **YOLO** in the technology stack. While U-Net delivers continuous pixel masks, industrial inspectors require discrete defect object counts and bounding boxes to tally flaws (e.g., counting discrete scratches, bubbles, or contamination spots).

12.2 Implementation Structure

- **Module:** [ai/models/object_detector.py](#) (`ObjectDetector`).
- **Underlying Model:** Ultralytics YOLO11n loaded from `ai/weights/yolo/baseline/weights/best.pt` .
- **Inference Configuration:**
 - Image size: `640 \times 640` pixels.
 - Confidence threshold: $\tau_{\text{conf}} = 0.25$.
 - IoU NMS threshold: $\tau_{\text{IoU}} = 0.70$.
 - Device: Automatic selection (`mps` , `cuda` , or `cpu`).
- **Output Data Structure:**

```
{
  "detected_objects_count": 5,
  "bounding_boxes": [
    {
      "x1": 99.62, "y1": 86.21, "x2": 809.85, "y2": 773.11,
      "confidence": 0.450, "class_id": 0, "class_name": "defect"
    },
    ...
  ]
}
```

13. Multi-Factor Severity & Quality Logic

When an image resolves as `DEFECTIVE` , the system calculates a multi-factor Severity Score using the official formula:

$$\text{Severity Score} = (0.30 \times \text{Size}) + (0.25 \times \text{Location}) + (0.25 \times \text{Defect Type}) + (0.20 \times \text{Confidence})$$

13.1 Scoring Components

1. **Defect Size Score (30%):** Evaluated in [ai/models/size_scorer.py](#) using percentile boundaries (`$p_{10}` to `$p_{95}`) from `size_score_boundaries.csv` .
2. **Defect Location Score (25%):** Evaluated in [ai/models/location_scorer.py](#) :
$$\text{Location Score} = 0.70 \times \left(1.0 - \frac{d_{\text{centroid}}}{d_{\text{max}}}\right) \times 100 + 0.30 \times \text{Area Factor}$$
3. **Defect Type Score (25%):** Looked up in [ai/models/defect_type_scorer.py](#) :
 - Structural fractures (`broken_large` , `crack` , `hole`): 90–95.
 - Surface flaws (`scratch` , `contamination` , `stain`): 40–70.
 - `unclassified_anomaly` : 70.0.
 - `normal` / `good` : 0.0.

4. **Detection Confidence (20%)**: Evaluated in [ai/models/confidence.py](#) via piecewise margin interpolation.

13.2 Severity Bands & Quality Decisions

- **Critical (80–100)**: Immediate quarantine and reject.
- **High (60–79.99)**: Defective component requiring repair or reject.
- **Medium (40–59.99)**: Moderate concern; manual inspection review required.
- **Low (0–39.99)**: Minor cosmetic flaw; acceptable under standard tolerance.

Quality Decision Rule: $\text{Decision} = \begin{cases} \text{Reject}, & \text{if Defective and Severity Score} \geq 60.0 \\ \text{Accept}, & \text{if Normal or Severity Score} < 60.0 \end{cases}$

14. Inspection API & Backend Integration

14.1 Key Endpoints ([backend/app/routers/image.py](#))

- `POST /images/{id}/inspect` : Executes end-to-end inspection, persists 17 fields into PostgreSQL, generates composite overlay PNG.
- `GET /images/{id}/inspection-overlay` : Streams visual overlay PNG with red contours and green bounding boxes.
- `GET /images/{id}` : Returns complete JSON serialization including `detected_objects_count` and `bounding_boxes` .

14.2 Database Schema Enhancements ([backend/app/models/image.py](#))

- `detected_objects_count` : `Column(Integer, default=0)`
 - `bounding_boxes` : `Column(JSON, nullable=True)`
-

15. Inspection Dashboard & Monitoring

The frontend ([frontend/src/pages/ImageDetails.jsx](#)) renders:

1. **Interactive Dual Viewer**: Side-by-side display of original raw image and composite defect overlay.
 2. **Defect Objects Indicator**: Displays `detected_objects_count` with an active `YOLO11n` badge.
 3. **Telemetry Table**: Category, defect subtype, anomaly score, detection confidence, predicted defect area, size score, location score, defect type score, severity score, and quality decision badge.
 4. **Visual Overlay Legend**: Informs users that red contours represent U-Net segmentation and green boxes represent YOLO11n defect instances.
-

16. YOLO Dataset Preparation

16.1 MVTec-to-YOLO Conversion

- **Source**: MVTec AD ground-truth segmentation masks (binary PNGs with pixel values $\{0, 255\}$).
- **Bounding Box Extraction**: Applied connected-component labeling with minimum contour area filtering (>10 pixels) to construct tight bounding boxes.
- **Coordinates**: Converted into normalized YOLO format: $\angle \text{class_id} \angle \angle x_{\text{center}} \angle \angle y_{\text{center}} \angle \angle \text{width} \angle \angle \text{height} \angle$.
- **Normal Specimen Handling**: Normal training and test images generated empty label text files (0 bounding boxes), strictly preserving the negative training signal.

16.2 Dataset Split Distribution

The generated dataset is stored at `datasets/mvtec_yolo/` :

Split	Defective Images	Normal Images	Total Images	Defect Bounding Boxes
Train	849	2,866	3,715	1,260
Validation	149	614	763	225
Test	260	616	876	403
Total	1,258	4,096	5,354	1,888

- **Average Boxes per Defective Image:** \$1.501\$.
 - **Multi-Box Defective Images:** 305 images (\$\approx 24.24\%\$).
 - **Source Integrity:** Zero alterations made to the original MVTec AD directory.
-

17. YOLO Experiments

Three controlled experiments were conducted and evaluated on the MVTec YOLO test split:

Experiment 1: Baseline Architecture (YOLO11n, 640x640)

- **Configuration:** YOLO11n, `imgsz=640` , epochs=50, batch=16, seed=42, optimizer=auto, device=mps.
- **Observation:**
 - Test Precision: **55.55%**
 - Test Recall: **39.95%**
 - Test mAP@50: **43.98%**
 - Test mAP@50-95: **20.19%**
- **Conclusion:** Effectively localizes large structural defects (e.g. cracked bottles, broken metal nuts), but misses subtle hairline scratches. Serves as our verified baseline.

Experiment 2: Increased Model Capacity (YOLO11s, 640x640)

- **Configuration:** YOLO11s, `imgsz=640` , epochs=50, batch=16, seed=42.
- **Observation:**
 - Test Precision: **41.64%**
 - Test Recall: **32.51%**
 - Test mAP@50: **31.64%**
 - Test mAP@50-95: **14.48%**
- **Conclusion:** Increasing model parameters from 2.6M to 9.4M caused severe overfitting on the small defective training set (849 defective samples). Model capacity scaling without additional data degrades test generalization.

Experiment 3: High-Resolution Industrial Tuning (YOLO11n, 1024x1024)

- **Configuration:** YOLO11n, `imgsz=1024` , industrial augmentations (mosaic disabled, reduced scale jitter).
- **Observation:**
 - Test Precision: **19.68%**
 - Test Recall: **17.12%**

- Test mAP@50: **11.80%**
 - Test mAP@50-95: **4.62%**
 - **Conclusion:** Higher spatial resolution reduced batch size and disturbed anchor-scale matching for small defects. Baseline YOLO11n at 640 remains superior.
-

18. YOLO Error Analysis & Diagnostics

To understand why the object detector misses certain defects, six targeted diagnostic evaluations were executed:

1. **Confidence Sweep Analysis:** Sweeping τ_{conf} from 0.25 down to 0.01 substantially increased candidate boxes but caused an explosion in false positives rather than recovering true defect locations.
 2. **Box-Level Confidence Distribution:** True defect boxes exhibited bimodal confidence—either strongly detected (>0.40) or missed entirely (<0.05).
 3. **IoU Quality Analysis:** Detected bounding boxes achieved high spatial alignment with ground truth ($\text{IoU} \approx 0.65 \text{--} 0.80$).
 4. **Defect-Size Dependency:**
 - Tiny defects ($<1\%$ of image area): Detection rate $<25\%$.
 - Large defects ($>5\%$ of image area): Detection rate $>80\%$.
 5. **Multi-Defect vs. Single-Defect Images:** Images containing multiple disjoint defects exhibited $\approx 18\%$ lower per-box recall than images with single isolated defects.
 6. **Inference Resolution Sweep:** Running inference with the trained 640 model at 768 and 1024 yielded no statistically meaningful gain in mAP, confirming that model representation, not input pixel scale, was the bounding constraint.
-

19. YOLO Integration into Production Pipeline

The verified YOLO11n baseline model was integrated into the production inspection pipeline with the following engineering design:

- **Location:** `ai/models/object_detector.py` integrated into `ai/models/inspection_pipeline.py`.
 - **Gated Execution:** YOLO runs strictly inside the `DEFECTIVE` branch. For `NORMAL` specimens (Quadrants A and D), YOLO execution is bypassed, eliminating false positives on nominal parts.
 - **Zero Decision Impact:** YOLO outputs do not alter Decision Fusion, severity formulas, or Accept/Reject outcomes.
 - **Dual Visual Overlay:** `create_defect_overlay()` renders both red U-Net contours and green YOLO bounding rectangles with confidence tags.
 - **Persistence & Frontend:** DB columns `detected_objects_count` and `bounding_boxes` are persisted in PostgreSQL and displayed in the React frontend table.
-

20. Evaluation Results

Bit-for-Bit Score Invariance Verification

Automated regression testing verified that integrating YOLO produced 100% bit-for-bit identical scores on all 14 inspection fields:

Field	Without YOLO	With Integrated YOLO	Match Status
predicted_category	bottle	bottle	PASS (Identical)
predicted_defect_type	broken_large	broken_large	PASS (Identical)
resolved_defect_status	broken_large	broken_large	PASS (Identical)
inspection_decision	DEFECTIVE	DEFECTIVE	PASS (Identical)
classification_confidence	50.34%	50.34%	PASS (Identical)
anomaly_score	2.3846006	2.3846006	PASS (Identical)
confidence_score	100.0%	100.0%	PASS (Identical)
predicted_area_percent	6.515067%	6.515067%	PASS (Identical)
size_score	58.947617	58.947617	PASS (Identical)
location_score	80.969734	80.969734	PASS (Identical)
defect_type_score	95.0	95.0	PASS (Identical)
severity_score	81.676718	81.676718	PASS (Identical)
severity_level	Critical	Critical	PASS (Identical)
quality_decision	Reject	Reject	PASS (Identical)

21. Challenges & Problems Encountered

- Extreme Class Imbalance:** The MVTec dataset provides 2,866 normal training images and only 849 defective images, causing standard object detectors to favor background predictions.
- Subtle Flaw Geometries:** Micro-defects (e.g. wire cuts or fine scratches) occupy fewer than 20\$ pixels, challenging anchor-based feature grids.
- Overfitting in Larger Architectures:** YOLO11s overfit significantly on the limited defective samples, demonstrating that larger models require extensive synthetic augmentation.
- False Positive Risks at Low Confidence:** Lowering confidence thresholds increased background false alarms without significantly improving genuine defect discovery.

22. Engineering Conclusions & Findings

Based on our empirical experiments under this dataset and configuration:

- Lowering confidence increased recall but produced many false positives.**
- Tiny defects were particularly difficult to localize using bounding boxes.**
- Multi-defect images were more difficult than single-defect images.**
- Increasing inference resolution did not materially improve the existing YOLO11n model.**
- The larger YOLO11s model did not outperform YOLO11n on the tested setup.**
- U-Net and YOLO provide different localization representations** (dense pixel mask vs discrete bounding boxes).
- U-Net remains the primary segmentation component** for physical defect area and severity scoring.

- 8. YOLO is retained as the object-detection/bounding-box branch for discrete defect counting and box visualization.
- 9. YOLO does NOT participate in Decision Fusion.
- 10. YOLO does NOT change the existing severity calculation.
- 11. YOLO does NOT replace the U-Net segmentation path.

23. Requirement-to-Implementation Traceability Matrix

Official Requirement	Current Implementation	Implementing File	Verification Evidence
Image Preprocessing Pipelines	\$224 \times 224\$ and \$640 \times 640\$ resizing, ImageNet norm, Pillow Bilinear resampling	ai/models/inspection_pipeline.py	Input shape validation, regression tests
Image Quality Analysis Reports	Defect area percentage, centroid location, size score, location score	ai/models/size_scorer.py, ai/models/location_scorer.py	Automated unit tests & regression logs
Image Analytics Workflows	Directed computational graph connecting classification, anomaly detection, U-Net, and YOLO	ai/models/inspection_pipeline.py	End-to-end inference verification
Train Defect Detection Models	ResNet18 Anomaly Detector, ResNet18 Defect Classifier, U-Net, YOLO11n	ai/models/	Trained model weights in ai/models/ and ai/weights.
Generate Defect Predictions	Autonomous category, defect subtype, anomaly score, severity, bounding boxes	ai/models/inspection_pipeline.py	verify_yolo_integration.p passing

Build Inspection Dashboards	Side-by-side inspection studio with dual overlay, metric cards, YOLO object counter	frontend/src/pages/ImageDetails.jsx	React 19 production build (npm run build PASS)
Object Detection (YOLO)	YOLO11n detector executed sequentially in defective branch	ai/models/object_detector.py	test_e2e_yolo_api.py passing
Defect Localization	Dense pixel U-Net mask + YOLO discrete bounding boxes	ai/models/defect_segmenter.py, ai/models/object_detector.py	Dual overlay PNG visual verification

24. Milestone 2 Verification Summary

- **Unit & Integration Tests:** 51 comprehensive tests passing in backend/tests/test_milestone2_comprehensive.py .
- **Regression Invariance Suite:** scratch/verify_yolo_integration.py passed with 100% bit-for-bit score matching.
- **End-to-End API & Database Suite:** scratch/test_e2e_yolo_api.py verified user authentication, upload, inference, PostgreSQL persistence, and overlay generation.
- **Client Build:** npm run build in frontend/ succeeded with 0 errors (606 modules transformed).

25. Milestone 3 Handover

With Milestone 2 verified, the system transitions directly into **Milestone 3 (Defect Classification & Manufacturing Analytics)**:

- Implementing defect categorization workflows and category-level aggregation.
- Establishing the Factory Supervisor review workflow with required audit notes.
- Generating downloadable Production Quality Reports in structured CSV format.
- Deploying the Manufacturing Analytics Dashboard with Recharts visualizations.
- Building Trend Monitoring workflows across 7-day, 30-day, and all-time horizons.
- Delivering advisory shop-floor Quality Recommendations (PASS , CLEAN , REWORK , SCRAP).